

ASSIGNMENT - 2

- 1) Use the dataset given in the link and analyze it to find the following statistics using spark.
- Find total number of lines and number of empty/malformed lines
 - Compute minimum, maximum and average sentence length after filtering data
 - Find most common tokens for at least 3 languages
 - Find TF-IDF vectors for text documents to measure the importance of words across Languages.
- Reference Dataset : <https://huggingface.co/datasets/ai4bharat/IndicCorpV2>

Ans -

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, length, explode
from pyspark.ml.feature import Tokenizer, HashingTF, IDF
from datasets import load_dataset

# -----
# Create Spark Session
# -----
spark = SparkSession.builder \
    .appName("IndicCorpV2_Q1_Analysis") \
    .getOrCreate()

# -----
# Load IndicCorpV2 Dataset (3 Languages)
# Dataset is huge, so we take a sample from each language
# -----
languages = ["hin_Deva", "tam_TamI", "ben_Beng"]
sample_per_language = 5000

data = []

for lang in languages:
    ds = load_dataset(
        "ai4bharat/IndicCorpV2",
        split=lang,
        streaming=True
    )

    for row in ds.take(sample_per_language):
        data.append((row["text"], lang))

# Create Spark DataFrame
df = spark.createDataFrame(data, ["text", "lang"])
```

```

# -----
# 1. Total Lines and Empty/Malformed Lines
# -----
total_lines = df.count()

empty_lines = df.filter(
    (col("text").isNull()) | (length(col("text")) == 0)
).count()

print("\n=====")
print("1. Line Statistics")
print("=====")
print("Total number of lines:", total_lines)
print("Empty/Malformed lines:", empty_lines)

# -----
# 2. Sentence Length Statistics (Min, Max, Avg)
# -----
filtered_df = df.filter(length(col("text")) > 0)

length_df = filtered_df.withColumn(
    "sent_length", length(col("text"))
)

min_len = length_df.agg({"sent_length": "min"}).collect()[0][0]
max_len = length_df.agg({"sent_length": "max"}).collect()[0][0]
avg_len = length_df.agg({"sent_length": "avg"}).collect()[0][0]

print("\n=====")
print("2. Sentence Length Statistics")
print("=====")
print("Minimum sentence length:", min_len)
print("Maximum sentence length:", max_len)
print("Average sentence length:", avg_len)

# -----
# 3. Most Common Tokens for 3 Languages
# -----
print("\n=====")
print("3. Most Common Tokens (Top 10)")
print("=====")

tokenizer = Tokenizer(inputCol="text", outputCol="tokens")

```

```

tokenized_df = tokenizer.transform(filtered_df)

for lang in languages:
    words_df = tokenized_df.filter(col("lang") == lang) \
        .select(explode(col("tokens")).alias("word"))

    # Remove empty tokens
    words_df = words_df.filter(col("word") != "")

    top_tokens = words_df.groupBy("word").count() \
        .orderBy(col("count").desc()) \
        .limit(10)

    print(f"\nLanguage: {lang}")
    top_tokens.show(truncate=False)

# -----
# 4. TF-IDF Vector Computation Across Documents
# -----
print("\n=====")
print("4. TF-IDF Vector Computation")
print("=====")

# Term Frequency (TF)
hashTF = HashingTF(
    inputCol="tokens",
    outputCol="rawFeatures",
    numFeatures=10000
)

tf_df = hashTF.transform(tokenized_df)

# Inverse Document Frequency (IDF)
idf = IDF(inputCol="rawFeatures", outputCol="tfidfFeatures")
idf_model = idf.fit(tf_df)

tfidf_df = idf_model.transform(tf_df)

print("\nTF-IDF vectors computed successfully!")
print("Total documents processed:", tfidf_df.count())
print("TF-IDF feature vector dimension: 10000")

# -----
# Stop Spark Session

```

```
# -----  
spark.stop()  
print("\n Spark Session Stopped Successfully")
```

```
(spark_env) user@DESKTOP-00TJMH0:~/bda_lab/Lab 2$ cat Q1.py  
from pyspark.sql import SparkSession  
from pyspark.sql.functions import col, length, explode  
from pyspark.ml.feature import Tokenizer, HashingTF, IDF  
from datasets import load_dataset  
  
# -----  
# Create Spark Session  
# -----  
spark = SparkSession.builder \  
    .appName("IndicCorpV2_Q1_Analysis") \  
    .getOrCreate()  
  
# -----  
# Load IndicCorpV2 Dataset (3 Languages)  
# Dataset is huge, so we take a sample from each language  
# -----  
languages = ["hin_Deva", "tam_Taml", "ben_Beng"]  
sample_per_language = 5000  
  
data = []  
  
for lang in languages:  
    ds = load_dataset(  
        "ai4bharat/IndicCorpV2",  
        split=lang,  
        streaming=True  
    )  
  
    for row in ds.take(sample_per_language):
```

```

        for row in ds.take(sample_per_language):
            data.append((row["text"], lang))

# Create Spark DataFrame
df = spark.createDataFrame(data, ["text", "lang"])

# -----
# 1. Total Lines and Empty/Malformed Lines
# -----
total_lines = df.count()

empty_lines = df.filter(
    (col("text").isNull()) | (length(col("text")) == 0)
).count()

print("\n=====")
print("1. Line Statistics")
print("=====")
print("Total number of lines:", total_lines)
print("Empty/Malformed lines:", empty_lines)

# -----
# 2. Sentence Length Statistics (Min, Max, Avg)
# -----
filtered_df = df.filter(length(col("text")) > 0)

length_df = filtered_df.withColumn(
    "sent_length", length(col("text"))
)

```

```

)

min_len = length_df.agg({"sent_length": "min"}).collect()[0][0]
max_len = length_df.agg({"sent_length": "max"}).collect()[0][0]
avg_len = length_df.agg({"sent_length": "avg"}).collect()[0][0]

print("\n=====")
print("2. Sentence Length Statistics")
print("=====")
print("Minimum sentence length:", min_len)
print("Maximum sentence length:", max_len)
print("Average sentence length:", avg_len)

# -----
# 3. Most Common Tokens for 3 Languages
# -----
print("\n=====")
print("3. Most Common Tokens (Top 10)")
print("=====")

tokenizer = Tokenizer(inputCol="text", outputCol="tokens")
tokenized_df = tokenizer.transform(filtered_df)

for lang in languages:
    words_df = tokenized_df.filter(col("lang") == lang) \
        .select(explode(col("tokens")).alias("word"))

    # Remove empty tokens
    words_df = words_df.filter(col("word") != "")

```

```

# Remove empty tokens
words_df = words_df.filter(col("word") != "")

top_tokens = words_df.groupBy("word").count() \
    .orderBy(col("count").desc()) \
    .limit(10)

print(f"\nLanguage: {lang}")
top_tokens.show(truncate=False)

# -----
# 4. TF-IDF Vector Computation Across Documents
# -----
print("\n=====")
print("4. TF-IDF Vector Computation")
print("=====")

# Term Frequency (TF)
hashTF = HashingTF(
    inputCol="tokens",
    outputCol="rawFeatures",
    numFeatures=10000
)

tf_df = hashTF.transform(tokenized_df)

# Inverse Document Frequency (IDF)
idf = IDF(inputCol="rawFeatures", outputCol="tfidfFeatures")
idf_model = idf.fit(tf_df)

```

```

# Inverse Document Frequency (IDF)
idf = IDF(inputCol="rawFeatures", outputCol="tfidfFeatures")
idf_model = idf.fit(tf_df)

tfidf_df = idf_model.transform(tf_df)

print("\nTF-IDF vectors computed successfully!")
print("Total documents processed:", tfidf_df.count())
print("TF-IDF feature vector dimension: 10000")

# -----
# Stop Spark Session
# -----
spark.stop()
print("\n✅ Spark Session Stopped Successfully")

```

Output :

```
(spark_env) user@DESKTOP-00TJMH0:~/bda_lab/Lab 2$ python Q1.py
WARNING: Using incubator modules: jdk.incubator.vector
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
26/01/31 11:58:05 WARN Utils: Your hostname, DESKTOP-00TJMH0, resolves to a loopback address: 127.0.1.1; using 10.255.25
5.254 instead (on interface lo)
26/01/31 11:58:05 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another address
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
26/01/31 11:58:06 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java cl
asses where applicable
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and
faster downloads.

=====
1. Line Statistics
=====
Total number of lines: 15000
Empty/Malformed lines: 7500

=====
2. Sentence Length Statistics
=====
Minimum sentence length: 5
Maximum sentence length: 41210
Average sentence length: 300.63706666666667
```

=====

3. Most Common Tokens (Top 10)

=====

Language: hin_Deva

word	count
के	5724
में	4334
की	3576
को	2712
से	2575
ने	2005
का	1909
औ	1858
है	1834
पर	1592

Language: tam_TamL

word	count
ஒரு	618
மற்றும்	563
இந்த	444
என்று	438

Language: tam_TamI

word	count
ஒரு	618
மற்றும்	563
இந்த	444
என்று	438
என	243
என்ற	239
அந்த	229
அவர்	219
-	202
இது	201

Language: ben_Beng

word	count
ও	1204
করে	816
থেকে	808
এই	746
এরং	731
করা	620
এ	570
জন্য	475
একটি	447

```
Language: ben_Beng
```

```
+-----+-----+
|word|count|
+-----+-----+
|ও|1204|
|কর|816|
|থেকে|808|
|এই|746|
|এবং|731|
|করা|620|
|এ|570|
|জ্য|475|
|একটি|447|
|তিনি|412|
+-----+-----+
```

```
=====
4. TF-IDF Vector Computation
=====
```

```
TF-IDF vectors computed successfully!
Total documents processed: 7500
TF-IDF feature vector dimension: 10000
```

```
Spark Session Stopped Successfully
(connected to spark-submit) [root@ip-172-31-1-101 ~]#
```

2) Use the amazon review dataset:

<https://www.kaggle.com/datasets/kritanjali/jain/amazon-reviews/data> for sentiment classification. Implement and train BiLSTM model with 70% train, 10% validation and 20% test split to predict the sentiment as positive or negative using PySpark. Use spark ML component for data preprocessing such as:

- Removal of punctuation and special characters
- Removal of stopwords
- Tokenize reviews

Evaluate your model performance by introducing noise in test split as follows:

- Randomly removing 20% words of each review
- Introduce 10% Spelling mistakes
- 20% Synonym replacement

Ans-

```
# =====
# Q2 Amazon Review Sentiment Classification using Spark + BiLSTM
# =====
```

```
from pyspark.sql import SparkSession
```

```

from pyspark.sql.functions import col, lower, regexp_replace, when
from pyspark.ml.feature import Tokenizer, StopWordsRemover

import numpy as np
import random

from tensorflow.keras.preprocessing.text import Tokenizer as KerasTokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, Bidirectional, LSTM, Dense

from sklearn.metrics import accuracy_score, classification_report

# =====
# 1. Spark Session
# =====

spark = SparkSession.builder \
    .appName("AmazonSentiment_BiLSTM_Q2") \
    .getOrCreate()

spark.sparkContext.setLogLevel("ERROR")

# =====
# 2. Load FULL Dataset
# =====

DATA_PATH = "archive
(1)/amazon_review_polarity_csv/amazon_review_polarity_csv/train.csv"

df = spark.read.csv(DATA_PATH, header=False, inferSchema=True)

df = df.select(
    col("_c0").alias("label"),
    col("_c2").alias("text")
)

print("\nTotal Reviews Loaded:", df.count())

# =====
# 3. Remove NULL or Empty Reviews (IMPORTANT FIX)

```

```

# =====

df = df.filter(col("text").isNotNull())
df = df.filter(col("text") != "")

# =====
# 4. Convert Labels → Binary Sentiment
# =====

df = df.withColumn(
    "label",
    when(col("label") == 2, 1).otherwise(0)
)

# =====
# 5. Spark ML Preprocessing
# =====

# Remove punctuation and special characters
df_clean = df.withColumn(
    "clean_text",
    lower(regexp_replace(col("text"), "[^a-zA-Z\\s]", ""))
)

# Tokenize
tokenizer = Tokenizer(inputCol="clean_text", outputCol="tokens")
df_tok = tokenizer.transform(df_clean)

# Stopword removal
remover = StopWordsRemover(inputCol="tokens", outputCol="filtered_tokens")
df_final = remover.transform(df_tok)

df_final = df_final.select("filtered_tokens", "label")

# =====
# 6. Train / Validation / Test Split
# =====

train_df, val_df, test_df = df_final.randomSplit([0.7, 0.1, 0.2], seed=42)

print("\nSplit Sizes:")

```

```

print("Train:", train_df.count())
print("Validation:", val_df.count())
print("Test:", test_df.count())

# =====
# 7. Convert Spark → Python (Subset for BiLSTM Training)
# =====

LIMIT = 30000

train_sample = train_df.limit(LIMIT)
val_sample = val_df.limit(5000)
test_sample = test_df.limit(5000)

train_rows = train_sample.collect()
val_rows = val_sample.collect()
test_rows = test_sample.collect()

train_texts = [" ".join(r.filtered_tokens) for r in train_rows]
val_texts = [" ".join(r.filtered_tokens) for r in val_rows]
test_texts = [" ".join(r.filtered_tokens) for r in test_rows]

train_labels = np.array([r.label for r in train_rows])
val_labels = np.array([r.label for r in val_rows])
test_labels = np.array([r.label for r in test_rows])

# =====
# 8. Prepare Sequences
# =====

max_words = 20000
max_len = 200

keras_tokenizer = KerasTokenizer(num_words=max_words)
keras_tokenizer.fit_on_texts(train_texts)

X_train = pad_sequences(keras_tokenizer.texts_to_sequences(train_texts),
                        maxlen=max_len)
X_val = pad_sequences(keras_tokenizer.texts_to_sequences(val_texts),
                     maxlen=max_len)
X_test = pad_sequences(keras_tokenizer.texts_to_sequences(test_texts),
                      maxlen=max_len)

```

```

# =====
# 9. BiLSTM Model
# =====

model = Sequential([
    Embedding(max_words, 128, input_length=max_len),
    Bidirectional(LSTM(64)),
    Dense(1, activation="sigmoid")
])

model.compile(loss="binary_crossentropy", optimizer="adam", metrics=["accuracy"])

print("\nTraining BiLSTM Model...\n")

model.fit(
    X_train, train_labels,
    validation_data=(X_val, val_labels),
    epochs=2,
    batch_size=64
)

# =====
# 10. Evaluation on Clean Test Set
# =====

pred_clean = (model.predict(X_test) > 0.5).astype("int")

print("\n=====")
print("CLEAN TEST PERFORMANCE")
print("=====")
print("Accuracy:", accuracy_score(test_labels, pred_clean))
print(classification_report(test_labels, pred_clean))

# =====
# 11. Noise Injection Functions
# =====

def remove_words(text, pct=0.2):
    words = text.split()
    k = int(len(words) * pct)

```

```

if len(words) > k and k > 0:
    words = random.sample(words, len(words) - k)
return " ".join(words)

def spelling_noise(text, pct=0.1):
    words = text.split()
    k = int(len(words) * pct)
    for i in random.sample(range(len(words)), min(k, len(words))):
        if len(words[i]) > 4:
            words[i] = words[i][: -1]
    return " ".join(words)

synonyms = {"good": "nice", "bad": "poor", "great": "excellent"}

def synonym_replace(text, pct=0.2):
    words = text.split()
    k = int(len(words) * pct)

    for i in random.sample(range(len(words)), min(k, len(words))):
        if words[i] in synonyms:
            words[i] = synonyms[words[i]]

    return " ".join(words)

# =====
# 12. Noisy Test Evaluation
# =====

noisy_texts = []

for t in test_texts:
    t = remove_words(t)
    t = spelling_noise(t)
    t = synonym_replace(t)
    noisy_texts.append(t)

X_noisy = pad_sequences(
    keras_tokenizer.texts_to_sequences(noisy_texts),
    maxlen=max_len
)

```

```

pred_noisy = (model.predict(X_noisy) > 0.5).astype("int")

print("\n=====")
print("NOISY TEST PERFORMANCE")
print("=====")
print("Accuracy:", accuracy_score(test_labels, pred_noisy))
print(classification_report(test_labels, pred_noisy))

```

```

# =====
# Stop Spark
# =====

```

```

spark.stop()
print("\nSpark Session Stopped Successfully")

```

```

(spark_env) user@DESKTOP-00TJMH0:~/bda_lab/Lab 2$ cat Q2.py
# =====
# Q2 Amazon Review Sentiment Classification using Spark + BiLSTM
# =====

from pyspark.sql import SparkSession
from pyspark.sql.functions import col, lower, regexp_replace, when
from pyspark.ml.feature import Tokenizer, StopWordsRemover

import numpy as np
import random

from tensorflow.keras.preprocessing.text import Tokenizer as KerasTokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, Bidirectional, LSTM, Dense

from sklearn.metrics import accuracy_score, classification_report

# =====
# 1. Spark Session
# =====

spark = SparkSession.builder \
    .appName("AmazonSentiment_BiLSTM_Q2") \
    .getOrCreate()

spark.sparkContext.setLogLevel("ERROR")

```

```

# =====
# 2. Load FULL Dataset
# =====

DATA_PATH = "archive (1)/amazon_review_polarity_csv/amazon_review_polarity_csv/train.csv"

df = spark.read.csv(DATA_PATH, header=False, inferSchema=True)

df = df.select(
    col("_c0").alias("label"),
    col("_c2").alias("text")
)

print("\nTotal Reviews Loaded:", df.count())

# =====
# 3. Remove NULL or Empty Reviews (IMPORTANT FIX)
# =====

df = df.filter(col("text").isNotNull())
df = df.filter(col("text") != "")

# =====
# 4. Convert Labels → Binary Sentiment
# =====

```

```

# =====
# 4. Convert Labels → Binary Sentiment
# =====

df = df.withColumn(
    "label",
    when(col("label") == 2, 1).otherwise(0)
)

# =====
# 5. Spark ML Preprocessing
# =====

# Remove punctuation and special characters
df_clean = df.withColumn(
    "clean_text",
    lower(regexp_replace(col("text"), "[^a-zA-Z\\s]", ""))
)

# Tokenize
tokenizer = Tokenizer(inputCol="clean_text", outputCol="tokens")
df_tok = tokenizer.transform(df_clean)

# Stopword removal
remover = StopWordsRemover(inputCol="tokens", outputCol="filtered_tokens")
df_final = remover.transform(df_tok)

df_final = df_final.select("filtered_tokens", "label")

```

```

# =====
# 6. Train / Validation / Test Split
# =====

train_df, val_df, test_df = df_final.randomSplit([0.7, 0.1, 0.2], seed=42)

print("\nSplit Sizes:")
print("Train:", train_df.count())
print("Validation:", val_df.count())
print("Test:", test_df.count())

# =====
# 7. Convert Spark → Python (Subset for BiLSTM Training)
# =====

LIMIT = 30000

train_sample = train_df.limit(LIMIT)
val_sample = val_df.limit(5000)
test_sample = test_df.limit(5000)

train_rows = train_sample.collect()
val_rows = val_sample.collect()
test_rows = test_sample.collect()

train_texts = [" ".join(r.filtered_tokens) for r in train_rows]
val_texts = [" ".join(r.filtered_tokens) for r in val_rows]
test_texts = [" ".join(r.filtered_tokens) for r in test_rows]

```

```

train_texts = [" ".join(r.filtered_tokens) for r in train_rows]
val_texts = [" ".join(r.filtered_tokens) for r in val_rows]
test_texts = [" ".join(r.filtered_tokens) for r in test_rows]

train_labels = np.array([r.label for r in train_rows])
val_labels = np.array([r.label for r in val_rows])
test_labels = np.array([r.label for r in test_rows])

# =====
# 8. Prepare Sequences
# =====

max_words = 20000
max_len = 200

keras_tokenizer = KerasTokenizer(num_words=max_words)
keras_tokenizer.fit_on_texts(train_texts)

X_train = pad_sequences(keras_tokenizer.texts_to_sequences(train_texts), maxlen=max_len)
X_val = pad_sequences(keras_tokenizer.texts_to_sequences(val_texts), maxlen=max_len)
X_test = pad_sequences(keras_tokenizer.texts_to_sequences(test_texts), maxlen=max_len)

# =====
# 9. BiLSTM Model
# =====

model = Sequential([

```

```
# =====
model = Sequential([
    Embedding(max_words, 128, input_length=max_len),
    Bidirectional(LSTM(64)),
    Dense(1, activation="sigmoid")
])

model.compile(loss="binary_crossentropy", optimizer="adam", metrics=["accuracy"])

print("\nTraining BiLSTM Model...\n")

model.fit(
    X_train, train_labels,
    validation_data=(X_val, val_labels),
    epochs=2,
    batch_size=64
)

# =====
# 10. Evaluation on Clean Test Set
# =====

pred_clean = (model.predict(X_test) > 0.5).astype("int")

print("\n=====")
print("CLEAN TEST PERFORMANCE")
print("=====")
print("Accuracy:", accuracy_score(test_labels, pred_clean))
```

```
print("\n=====")
print("CLEAN TEST PERFORMANCE")
print("=====")
print("Accuracy:", accuracy_score(test_labels, pred_clean))
print(classification_report(test_labels, pred_clean))

# =====
# 11. Noise Injection Functions
# =====

def remove_words(text, pct=0.2):
    words = text.split()
    k = int(len(words) * pct)
    if len(words) > k and k > 0:
        words = random.sample(words, len(words) - k)
    return " ".join(words)

def spelling_noise(text, pct=0.1):
    words = text.split()
    k = int(len(words) * pct)
    for i in random.sample(range(len(words)), min(k, len(words))):
        if len(words[i]) > 4:
            words[i] = words[i][:-1]
    return " ".join(words)

synonyms = {"good": "nice", "bad": "poor", "great": "excellent"}
```

```

synonyms = {"good": "nice", "bad": "poor", "great": "excellent"}

def synonym_replace(text, pct=0.2):
    words = text.split()
    k = int(len(words) * pct)

    for i in random.sample(range(len(words)), min(k, len(words))):
        if words[i] in synonyms:
            words[i] = synonyms[words[i]]

    return " ".join(words)

# =====
# 12. Noisy Test Evaluation
# =====

noisy_texts = []

for t in test_texts:
    t = remove_words(t)
    t = spelling_noise(t)
    t = synonym_replace(t)
    noisy_texts.append(t)

X_noisy = pad_sequences(
    keras_tokenizer.texts_to_sequences(noisy_texts),
    maxlen=max_len

```

```

X_noisy = pad_sequences(
    keras_tokenizer.texts_to_sequences(noisy_texts),
    maxlen=max_len
)

pred_noisy = (model.predict(X_noisy) > 0.5).astype("int")

print("\n=====")
print("NOISY TEST PERFORMANCE")
print("=====")
print("Accuracy:", accuracy_score(test_labels, pred_noisy))
print(classification_report(test_labels, pred_noisy))

# =====
# Stop Spark
# =====

spark.stop()
print("\nSpark Session Stopped Successfully")

(spark_env) user@DESKTOP-00TJMH0:~/bda_lab/Lab 2$ |

```

Output :

```
(spark_env) user@DESKTOP-00TJMH0:~/bda_lab/Lab 2$ ls
Q1.py  Q2.py  'archive (1)'  'archive (1).zip'  'archive (1).zip:Zone.Identifier'
(spark_env) user@DESKTOP-00TJMH0:~/bda_lab/Lab 2$ python Q2.py
2026-01-31 10:47:29.721863: I external/local_xla/xla/tsl/cuda/cudart_stub.cc:31] Could not find cuda drivers on your
hine, GPU will not be used.
2026-01-31 10:47:29.744700: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly
fferent numerical results due to floating-point round-off errors from different computation orders. To turn them off,
t the environment variable 'TF_ENABLE_ONEDNN_OPTS=0'.
2026-01-31 10:47:30.417125: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized
use available CPU instructions in performance-critical operations.
To enable the following instructions: AVX2 AVX_VNNI FMA, in other operations, rebuild TensorFlow with the appropriate
mpiler flags.
2026-01-31 10:47:32.829763: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly
fferent numerical results due to floating-point round-off errors from different computation orders. To turn them off,
t the environment variable 'TF_ENABLE_ONEDNN_OPTS=0'.
2026-01-31 10:47:32.834319: I external/local_xla/xla/tsl/cuda/cudart_stub.cc:31] Could not find cuda drivers on your
hine, GPU will not be used.
WARNING: Using incubator modules: jdk.incubator.vector
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
26/01/31 10:47:36 WARN Utils: Your hostname, DESKTOP-00TJMH0, resolves to a loopback address: 127.0.1.1; using 10.255
5.254 instead (on interface lo)
26/01/31 10:47:36 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another address
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
26/01/31 10:47:37 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java
asses where applicable

Total Reviews Loaded: 3600000
```

```
Total Reviews Loaded: 3600000

Split Sizes:
Train: 2520607
Validation: 360103
Test: 719277
/home/user/bda_lab/spark_env/lib/python3.12/site-packages/keras/src/layers/core/embedding.py:100: UserWarning: Argument
`input_length` is deprecated. Just remove it.
  warnings.warn(
2026-01-31 10:54:44.253377: E external/local_xla/xla/stream_executor/cuda/cuda_platform.cc:51] failed call to cuInit: IN
TERNAL: CUDA error: Failed call to cuInit: UNKNOWN ERROR (303)

Training BiLSTM Model...

Epoch 1/2
469/469 ██████████ 98s 203ms/step - accuracy: 0.7871 - loss: 0.4402 - val_accuracy: 0.8368 - val_loss: 0.3721
Epoch 2/2
469/469 ██████████ 100s 212ms/step - accuracy: 0.8943 - loss: 0.2591 - val_accuracy: 0.8368 - val_loss: 0.3850
157/157 ██████████ 9s 55ms/step

=====
CLEAN TEST PERFORMANCE
=====
Accuracy: 0.8246


|          | precision | recall | f1-score | support |
|----------|-----------|--------|----------|---------|
| 0        | 0.81      | 0.81   | 0.81     | 2337    |
| 1        | 0.83      | 0.84   | 0.84     | 2663    |
| accuracy |           |        | 0.82     | 5000    |


```

```
=====
CLEAN TEST PERFORMANCE
=====
Accuracy: 0.8246
      precision    recall  f1-score   support

         0         0.81      0.81      0.81      2337
         1         0.83      0.84      0.84      2663

   accuracy          0.82          0.82          0.82          5000
  macro avg          0.82          0.82          0.82          5000
weighted avg          0.82          0.82          0.82          5000

157/157 ██████████ 7s 46ms/step
=====
```

```
=====
NOISY TEST PERFORMANCE
=====
Accuracy: 0.8016
      precision    recall  f1-score   support

         0         0.79      0.79      0.79      2337
         1         0.81      0.81      0.81      2663

   accuracy          0.80          0.80          0.80          5000
  macro avg          0.80          0.80          0.80          5000
weighted avg          0.80          0.80          0.80          5000
```

Spark Session Stopped Successfully